

Task 1: Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```

00000000,00000000,00000000,00000000,00000000,00000000,00000000,00000000
Mems_allowed_list: 0
voluntary_ctxt_switches: 43
nonvoluntary_ctxt_switches: 1
saikrishna@DESKTOP-LGFVDTJ:~$ nano practicalweek4.c
saikrishna@DESKTOP-LGFVDTJ:~$ gcc practicalweek4.c -o practicalweek4
ccl: fatal error: practicalweek4.c: No such file or directory
compilation terminated.
saikrishna@DESKTOP-LGFVDTJ:~$ gcc practicalweek4.c -o practicalweek4
saikrishna@DESKTOP-LGFVDTJ:~$ ./practicalweek4
Parent PID: 1024
Child 1: PID = 1025
Child 2: PID = 1026

Using wait():
Child 3: PID = 1027
Child 1 finished
wait() collected child PID: 1025
Child 2 finished
wait() collected child PID: 1026
Child 3 finished
wait() collected child PID: 1027
Child 1: PID = 1028

Child 2: PID = 1029
Using waitpid():
Child 3: PID = 1030
Child 1 finished
waitpid() collected child PID: 1028
Child 2 finished
waitpid() collected child PID: 1029
Child 3 finished
waitpid() collected child PID: 1030
Parent process completed.
saikrishna@DESKTOP-LGFVDTJ:~$ cat practicalweek4.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int i;
    pid_t pid;
    pid_t child[3];

    printf("Parent PID: %d\n", getpid());

    // Create 3 child processes
    for (i = 0; i < 3; i++) {
        pid = fork();

        if (pid < 0) {
            printf("Fork failed\n");
            return 1;
        }
    }
}

```

```

    printf("Fork failed\n");
    return 1;
}

if (pid == 0) {
    printf("Child %d: PID = %d\n", i + 1, getpid());
    sleep(2 + i);
    printf("Child %d finished\n", i + 1);
    exit(0);
}

child[i] = pid;
}

// Using wait()
printf("\nUsing wait():\n");

for (i = 0; i < 3; i++) {
    pid = wait(NULL);
    printf("wait() collected child PID: %d\n", pid);
}

// Create another 3 children
for (i = 0; i < 3; i++) {
    pid = fork();

    if (pid == 0) {
        printf("Child %d: PID = %d\n", i + 1, getpid());
        sleep(2 + i);
        printf("Child %d finished\n", i + 1);
        exit(0);
    }

    child[i] = pid;
}

// Using waitpid()
printf("\nUsing waitpid():\n");

for (i = 0; i < 3; i++) {
    waitpid(child[i], NULL, 0);
    printf("waitpid() collected child PID: %d\n", child[i]);
}

printf("Parent process completed.\n");

return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ -|

```

Task 2: Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

Parent exiting
saikrishna@DESKTOP-LGFVDTJ:~$ ps -o pid,ppid,state,cmd
  PID   PPID S CMD
   329    324 S -bash
  1173    329 R ps -o pid,ppid,state,cmd
saikrishna@DESKTOP-LGFVDTJ:~$ cat Pacticalweek4.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child process running\n");
        printf("Child PID: %d\n", getpid());

        sleep(2);

        printf("Child exiting\n");
        exit(0);
    }
    else {
        printf("Parent process running\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child completed successfully\n");
        printf("Parent exiting\n");
    }

    return 0;
}
saikrishna@DESKTOP-LGFVDTJ:~$ ./Pacticalweek4
Parent process running
Parent PID: 1230
Child process running
Child PID: 1231
Child exiting
Child completed successfully
Parent exiting
saikrishna@DESKTOP-LGFVDTJ:~$ ^C
saikrishna@DESKTOP-LGFVDTJ:~$ wait(NULL);
-bash: syntax error near unexpected token `NULL'
saikrishna@DESKTOP-LGFVDTJ:~$ |

```